

NexusDashboard.Connect.ps1

PowerShell script - reads the switches managed by Cisco Nexus Dashboard (Fabric Controller) over its REST API, logging in through the TACACS login domain, and adds them to SolarWinds as nodes with their pollers and interfaces, the way DNA.DiscoverNodesAndInterfaces.ps1 does it for Cisco DNA Center. Every run is logged.

762 lines · Scripts/NexusDashboard.Connect.ps1

```
1 #Requires -Version 7
2
3 <#
4 .SYNOPSIS
5     SolarWinds SWIS PowerShell script add the switches managed by Cisco Nexus Dashboard as nodes and
6     discover their interfaces
7
8 .DESCRIPTION
9     This script combines the Cisco Nexus Dashboard (ND) REST API with two
10    building blocks from the SolarWinds SDK samples, the same way
11    DNA.DiscoverNodesAndInterfaces.ps1 does it for Cisco DNA Center:
12
13    o Nexus Dashboard REST API
14      - authenticates (POST /login) against the login domain given in
15        -NXDomain and keeps the returned JWT for follow-up calls;
16      - refreshes the token (POST /refresh) before it expires - ND tokens
17        are valid for 20 minutes by default;
18      - reads the switch inventory managed by the Fabric Controller (NDFC)
19        service;
20      - logs out (POST /logout) as soon as the inventory is read, so the
21        token is revoked rather than left valid while the nodes are added.
22
23    o CRUD.AddNode.ps1
24      - adds a node <component> using CRUD operations (New-SwisObject) and
25        registers the standard set of pollers for it (Status, Response
26        Time, Details, Uptime).
27
28    o NPM.DiscoverAndAddInterfacesOnNode.ps1
29      - uses Orion.NPM.Interfaces.DiscoverInterfacesOnNode and
30        Orion.NPM.Interfaces.AddInterfacesOnNode (SWISv3 verbs, NPM only)
31        to discover and add the interfaces of a node.
32
33    The result is a single script that walks the switches Nexus Dashboard
34    reports, adds every one of them that does not already exist in
35    SolarWinds by its management IP address - with the switch name as the
36    node caption - and then discovers and adds their interfaces for
37    monitoring. -InterfaceFilter decides which interfaces are kept.
38
39    Every run records each switch returned, each component that was added,
40    already existed, was skipped or failed, every interface that was added,
41    and every poller that was registered, followed by a summary line.
42
43    Please update the parameter defaults below to match your environment
44    before running.
45
46 .PARAMETER NXServer
47     Host name or IP address of the Nexus Dashboard (the cluster's management
48     address). The API is reached on the default HTTPS port (443).
49
50 .PARAMETER NXCredentialPath
51     Path to a credential saved with Export-Clixml, for unattended runs
52     (scheduled task). Pass an empty string to be prompted instead. Create
53     the file once, as the account the task runs under:
54     Get-Credential | Export-Clixml -Path D:
55     \SolarWindsScripts\PowerShell\Credentials\NexusDashboard_Credential.xml
56
57 .PARAMETER NXDomain
```

```

56 The ND login domain the user authenticates against. "TACACS" is the
57 TACACS+ login domain; it must match the domain name configured in ND
58 (Admin > Authentication > Login Domains) exactly, case included. Use
59 "DefaultAuth" for a user from the local ND user database.
60
61 .PARAMETER TokenRefreshMinutes
62     Age, in minutes, after which the ND token is renewed before the next
63     call. Must stay below the token lifetime configured in ND (20 minutes by
64     default).
65
66 .PARAMETER SwisHost
67     Address of the SolarWinds server (SWIS).
68
69 .PARAMETER SwisCredentialPath
70     Path to a PScredential exported with Export-Clixml for a SolarWinds user
71     that can add nodes.
72
73 .PARAMETER EngineID
74     SolarWinds polling engine the new nodes are assigned to.
75
76 .PARAMETER SNMPVersion
77     SNMP version used to poll the new nodes.
78
79 .PARAMETER SNMPCommunity
80     SNMP community configured on the switches. Without it the Details/Uptime
81     pollers and the interface discovery have nothing to poll.
82
83 .PARAMETER InterfaceFilter
84     Regular expression matched against the interface caption. Only the
85     interfaces that match it and are operationally up (ifOperStatus 1) are
86     added. The default keeps the NX-OS physical ports and port-channels
87     (Ethernet1/1, Eth1/1, port-channel10, Po10); everything else - mgmt0,
88     Vlan, loopback, nve and the like - is left out. The match is not case
89     sensitive.
90
91 .PARAMETER LogPath
92     The log file this script writes to. It is bounded by rotation rather
93     than by age, the same way DNA.DiscoverNodesAndInterfaces.ps1 bounds its
94     own log.
95
96 .PARAMETER LogMaxBytes
97     The size at which the log is rotated. Rotation keeps LogKeep older
98     generations beside it, so the space the logging can occupy is bounded at
99     roughly LogMaxBytes times LogKeep plus one.
100
101 .PARAMETER LogKeep
102     How many rotated logs to keep. Zero keeps none: the log starts over
103     instead.
104
105 .EXAMPLE
106     .\NexusDashboard.Connect.ps1
107
108     Runs with the defaults set in the param block.
109
110 .EXAMPLE
111     .\NexusDashboard.Connect.ps1 -NXServer nd.example.local -SwisHost orion.example.local `
112     -SNMPCommunity "mycommunity"
113
114 .NOTES
115     Requires PowerShell 7+ for the -SkipCertificateCheck switch on
116     Invoke-RestMethod, and the SwisPowerShell module for Connect-Swis.
117
118     The certificate of the Nexus Dashboard is not validated: ND ships with a
119     self-signed certificate, so Connect-NexusDashboard is always called with
120     -TrustAllCertificates, the same way FMC.DiscoverNodes.ps1 calls

```

```

121 Connect-Fmc. Once ND carries a certificate the server trusts, drop that
122 switch from the call so the certificate is checked again.
123 #>
124
125 param(
126     # --- Cisco Nexus Dashboard connection ---
127     [string]$NXServer = "1.1.1.1",
128     [string]$NXCredentialPath = "D:\SolarWindsScripts\PowerShell\Credentials\FMC_Credential.xml",
129     [string]$NXDomain = "TACACS",
130
131     [ValidateRange(1, 19)]
132     [int]$TokenRefreshMinutes = 15,
133
134     # --- SolarWinds connection ---
135     [string]$SwisHost = "5.5.5.5",
136     [string]$SwisCredentialPath = "D:\SolarWindsScripts\PowerShell\Credentials\SolarWindsCredentials.xml",
137
138     # Shared settings applied to every switch retrieved from Nexus Dashboard.
139     [int]$EngineID = 2,
140     [int]$SNMPVersion = 2,
141     [string]$SNMPCommunity = '',
142     [string]$InterfaceFilter = '^(Ethernet|Eth\d|port-channel|Po\d)',
143
144     # --- Log ---
145     [string] $LogPath = (Join-Path -Path $env:ProgramData -ChildPath 'SolarWinds\nexusdashboard-discover-
146     nodes.log'),
147
148     [ValidateRange(4KB, 100MB)]
149     [int] $LogMaxBytes = 1MB,
150
151     [ValidateRange(0, 20)]
152     [int] $LogKeep = 3
153 )
154 # --- Logging ---
155 # Taken from DNA.DiscoverNodesAndInterfaces.ps1: one log file bounded by
156 # size-based rotation, the same level set and line format, and the same lazy
157 # creation of the log directory on first write. The file calls carry
158 # -ErrorAction Stop of their own, since this script does not set
159 # $ErrorActionPreference = 'Stop'; without it a failed write escapes the
160 # catch as a raw error instead of the intended warning.
161
162 function Invoke-LogRotation {
163     # Renames the log out of the way once it reaches -LogMaxBytes, keeping
164     # -LogKeep older generations: .log -> .log.1, .log.1 -> .log.2, and so on,
165     # with the oldest dropped. Run on a schedule the log would otherwise grow
166     # without bound.
167     param(
168         [Parameter(Mandatory)][string] $Path
169     )
170
171     if (-not (Test-Path -LiteralPath $Path)) { return }
172     if ((Get-Item -LiteralPath $Path).Length -lt $script:LogMaxBytes) { return }
173
174     # Nothing to keep: the log simply starts over.
175     if ($script:LogKeep -lt 1) {
176         Remove-Item -LiteralPath $Path -Force -WhatIf:$false -Confirm:$false -ErrorAction Stop
177         return
178     }
179
180     # Oldest generation falls off the end.
181     $oldest = "$Path.$($script:LogKeep)"
182     if (Test-Path -LiteralPath $oldest) {
183         Remove-Item -LiteralPath $oldest -Force -WhatIf:$false -Confirm:$false -ErrorAction Stop
184     }

```

```

185
186 # Shift the rest down, highest first so nothing is overwritten on the way.
187 for ($i = $script:LogKeep - 1; $i -ge 1; $i--) {
188     $from = "$Path.$i"
189     if (Test-Path -LiteralPath $from) {
190         Move-Item -LiteralPath $from -Destination "$Path.($i + 1)" -Force -WhatIf:$false -Confirm:$false
191     }
192 }
193
194 Move-Item -LiteralPath $Path -Destination "$Path.1" -Force -WhatIf:$false -Confirm:$false -ErrorAction
195 Stop
196
197 function Write-Log {
198     <#
199     .SYNOPSIS
200         Record one line in the run log, and on the console unless suppressed
201     .PARAMETER NoConsole
202         Write to the log file only. Used for the per-item detail lines - every
203         switch, component, interface and poller - which belong in the record
204         but would bury the console summary.
205     #>
206     param(
207         [Parameter(Mandatory)]
208         [AllowEmptyString()]
209         [string] $Message,
210
211         [ValidateSet('INFO', 'WARN', 'ERROR')]
212         [string] $Level = 'INFO',
213
214         [switch] $NoConsole
215     )
216
217     $line = '{0:yyyy-MM-dd HH:mm:ss} [{1,-5}] {2}' -f (Get-Date), $Level, $Message
218     if (-not $NoConsole) {
219         Write-Information -MessageData $line -InformationAction Continue
220     }
221
222     # -WhatIf:$false so that the log keeps being written even if a caller
223     # passes -WhatIf through; the log is a record, never one of the changes
224     # being previewed.
225     try {
226         $directory = Split-Path -Path $script:LogPath -Parent
227         if ($directory -and -not (Test-Path -LiteralPath $directory)) {
228             New-Item -Path $directory -ItemType Directory -Force -WhatIf:$false -Confirm:$false -ErrorAction
229             Stop | Out-Null
230         }
231         Invoke-LogRotation -Path $script:LogPath
232         Add-Content -LiteralPath $script:LogPath -Value $line -Encoding UTF8 -WhatIf:$false -Confirm:$false -
233         ErrorAction Stop
234     } catch {
235         Write-Warning "Could not write to the log file '$script:LogPath': $($_.Exception.Message)"
236     }
237 }
238
239 function Get-NexusDashboardErrorMessage {
240     # Turns a failed Invoke-RestMethod into one readable line: the HTTP status
241     # plus the error text ND puts in the body, when there is one.
242     param(
243         [Parameter(Mandatory = $true)] [System.Management.Automation.ErrorRecord]$ErrorRecord
244     )
245
246     $status = $null

```

```

246 if ($ErrorRecord.Exception.Response) {
247     $status = [int]$ErrorRecord.Exception.Response.StatusCode
248 }
249
250 $detail = $ErrorRecord.ErrorDetails.Message
251 if (-not $detail) { $detail = $ErrorRecord.Exception.Message }
252
253 if ($status) { "HTTP ${status}: $detail" } else { $detail }
254}
255
256 function Connect-NexusDashboard {
257     param(
258         [Parameter(Mandatory = $true)] [string]$Server,
259         [Parameter(Mandatory = $true)] [string]$Username,
260         [Parameter(Mandatory = $true)] [System.Security.SecureString]$Password,
261         # Login domain. "DefaultAuth" is the local user database; for a remote
262         # (RADIUS/TACACS/LDAP) user pass the login domain name configured in ND.
263         [string]$Domain = "DefaultAuth",
264         [switch]$TrustAllCertificates,
265         # Renew the token once it is older than this, ahead of the 20 minute
266         # default expiry.
267         [int]$TokenRefreshMinutes = 15
268     )
269
270     # PtrToStringBSTR (rather than PtrToStringAuto) decodes the BSTR as UTF-16 on
271     # every platform, and the buffer is zeroed again right after it is read.
272     $passwordPtr = [System.Runtime.InteropServices.Marshal]::SecureStringToBSTR($Password)
273     try {
274         $plainPassword = [System.Runtime.InteropServices.Marshal]::PtrToStringBSTR($passwordPtr)
275     }
276     finally {
277         [System.Runtime.InteropServices.Marshal]::ZeroFreeBSTR($passwordPtr)
278     }
279
280     $baseUri = "https://$Server"
281
282     $body = @{
283         userName    = $Username
284         userPasswd  = $plainPassword
285         domain      = $Domain
286     } | ConvertTo-Json
287
288     $params = @{
289         Uri          = "$baseUri/login"
290         Method       = "Post"
291         Body         = $body
292         ContentType  = "application/json"
293         SessionVariable = "webSession"
294     }
295     if ($TrustAllCertificates) {
296         $params["SkipCertificateCheck"] = $true
297     }
298
299     try {
300         $response = Invoke-RestMethod @params
301     }
302     catch {
303         throw "Login to $Server failed: $(Get-NexusDashboardErrorMessage -ErrorRecord $_)"
304     }
305
306     # The token is returned as "jwttoken" (older releases also mirror it in
307     # "token") and set as the AuthCookie cookie; any of the three will do.
308     $token = $response.jwttoken
309     if (-not $token) { $token = $response.token }
310     if (-not $token) {

```

```

311     $cookie = $webSession.Cookies.GetCookies($baseUri) |
312         Where-Object { $_.Name -eq "AuthCookie" } |
313         Select-Object -First 1
314     $token = $cookie.Value
315 }
316 if (-not $token) {
317     throw "Login to $Server succeeded but no token was returned."
318 }
319
320 [pscustomobject]@{
321     Server          = $Server
322     BaseUri         = $baseUri
323     Username        = $Username
324     Token           = $token
325     IssuedAt        = Get-Date
326     TokenRefreshMinutes = $TokenRefreshMinutes
327     TrustAllCertificates = [bool]$TrustAllCertificates
328 }
329 }
330
331 function Update-NexusDashboardToken {
332     # ND tokens are short lived (20 minutes by default), so a long run - such as
333     # walking a large switch inventory - has to renew the token before it lapses.
334     # The refreshed token replaces the old one on the session object in place.
335     param(
336         [Parameter(Mandatory = $true)] [pscustomobject]$Session
337     )
338
339     $params = @{
340         Uri      = "$($Session.BaseUri)/refresh"
341         Method    = "Post"
342         Headers   = @{ "Authorization" = "Bearer $($Session.Token)" }
343         ContentType = "application/json"
344     }
345     if ($Session.TrustAllCertificates) {
346         $params["SkipCertificateCheck"] = $true
347     }
348
349     try {
350         $response = Invoke-RestMethod @params
351     }
352     catch {
353         throw "Token refresh on $($Session.Server) failed: $(Get-NexusDashboardErrorMessage -ErrorRecord $_)"
354     }
355
356     $token = $response.jwttoken
357     if (-not $token) { $token = $response.token }
358     if (-not $token) {
359         throw "Token refresh on $($Session.Server) returned no token."
360     }
361
362     $Session.Token = $token
363     $Session.IssuedAt = Get-Date
364 }
365
366 function Invoke-NexusDashboardApi {
367     param(
368         [Parameter(Mandatory = $true)] [pscustomobject]$Session,
369         [Parameter(Mandatory = $true)] [string]$Path,
370         [ValidateSet("Get", "Post", "Put", "Delete")] [string]$Method = "Get",
371         $Body
372     )
373
374     if (((Get-Date) - $Session.IssuedAt).TotalMinutes -ge $Session.TokenRefreshMinutes) {
375         Update-NexusDashboardToken -Session $Session

```

```

376     Write-Log "Token refreshed on $($Session.Server)."
377 }
378
379 $params = @{
380     Uri      = "$($Session.BaseUri)/$($Path.TrimStart('/'))"
381     Method   = $Method
382     Headers  = @{ "Authorization" = "Bearer $($Session.Token)" }
383 }
384 if ($null -ne $Body) {
385     $params["Body"] = if ($Body -is [string]) { $Body } else { $Body | ConvertTo-Json -Depth 10 }
386     $params["ContentType"] = "application/json"
387 }
388 if ($Session.TrustAllCertificates) {
389     $params["SkipCertificateCheck"] = $true
390 }
391
392 try {
393     Invoke-RestMethod @params
394 }
395 catch {
396     throw "$Method $Path on $($Session.Server) failed: $(Get-NexusDashboardErrorMessage -ErrorRecord $_)"
397 }
398 }
399
400 function Disconnect-NexusDashboard {
401     param(
402         [Parameter(Mandatory = $true)] [pscustomobject]$Session
403     )
404
405     # Logging out invalidates the token right away instead of leaving it valid
406     # until it expires. A failure here is not worth failing the run over.
407     #
408     # Nexus Dashboard 3.2 answers POST /logout with 405 (Method Not Allowed),
409     # so GET is tried next. If neither is accepted there is nothing more to
410     # do: the token simply expires on its own after the ND session timeout
411     # (20 minutes by default), the same way the DNA token is left to expire.
412     foreach ($method in @("Post", "Get")) {
413         $params = @{
414             Uri      = "$($Session.BaseUri)/logout"
415             Method   = $method
416             Headers  = @{ "Authorization" = "Bearer $($Session.Token)" }
417         }
418         if ($Session.TrustAllCertificates) {
419             $params["SkipCertificateCheck"] = $true
420         }
421
422         try {
423             Invoke-RestMethod @params | Out-Null
424             Write-Log "Logged out of $($Session.Server) $($method.ToUpper()) /logout)."
425             return
426         }
427         catch {
428             $status = $null
429             if ($_.Exception.Response) {
430                 $status = [int]$_ .Exception.Response.StatusCode
431             }
432             if ($status -eq 405) { continue }
433
434             Write-Log "Logout from $($Session.Server) failed: $(Get-NexusDashboardErrorMessage -ErrorRecord
435 $_)" -Level WARN
436             return
437         }
438     }
439 }

```

```

Write-Log "Nexus Dashboard on $($Session.Server) accepts no logout request - the token is left to expire
on its own."
440}
441
442# --- Function: Read the switch inventory from Nexus Dashboard ---
443function Get-NexusDashboardSwitches {
444    <#
445    .SYNOPSIS
446        Get the switches managed by the Fabric Controller (NDFC) service
447    .PARAMETER Session
448        The session returned by Connect-NexusDashboard
449    .OUTPUTS
450        One object per switch: Name, IPAddress, Model, Serial, Fabric, Role,
451        Version and Status
452    #>
453    param(
454        [Parameter(Mandatory = $true)] [pscustomobject]$Session
455    )
456
457    # The response is a JSON array; @() unrolls it whether it arrives as one
458    # array object or one switch at a time.
459    $rawSwitches = Invoke-NexusDashboardApi -Session $Session `
460        -Path "/appcenter/cisco/ndfc/api/v1/lan-fabric/rest/inventory/allswitches"
461
462    @($rawSwitches) | Where-Object { $_ } | ForEach-Object {
463        [pscustomobject]@{
464            Name      = $_.logicalName
465            IPAddress = $_.ipAddress
466            Model     = $_.model
467            Serial    = $_.serialNumber
468            Fabric    = $_.fabricName
469            Role      = $_.switchRole
470            Version   = $_.release
471            Status    = $_.status
472        }
473    }
474}
475
476# --- Function: End the Nexus Dashboard and SWIS sessions ---
477function Stop-Sessions {
478    <#
479    .SYNOPSIS
480        End the Nexus Dashboard and SWIS sessions opened by this script
481    .DESCRIPTION
482        Nexus Dashboard publishes a logout endpoint, so the token is revoked
483        rather than left to expire.
484
485        SWIS has no Disconnect-Swis cmdlet, but Connect-Swis returns an
486        InfoServiceProxy, which is IDisposable, so the connection can be
487        closed explicitly instead of being left to the finalizer.
488
489        Safe to call more than once - each session is ended only once.
490    #>
491    if ($script:ndSession) {
492        Disconnect-NexusDashboard -Session $script:ndSession
493        $script:ndSession = $null
494    }
495
496    if ($script:swis) {
497        try {
498            $script:swis.Close()
499            Write-Log "Closed the SWIS connection."
500        } catch {
501            Write-Log "Failed to close the SWIS connection: $($_.Exception.Message)" -Level WARN
502        }
503    }
504}

```

```

503     $script:swis = $null
504 }
505}
506
507# --- Counters for the summary at the end of the run ---
508$script:stats = [ordered]@{
509     ComponentsAdded      = 0
510     ComponentsExisting   = 0
511     ComponentsSkipped    = 0
512     ComponentsFailed     = 0
513     InterfacesAdded      = 0
514     PollersAdded         = 0
515}
516
517Write-Log "===== Run started: Nexus Dashboard $NXServer -> SolarWinds $SwisHost ====="
518
519# --- Connect to Nexus Dashboard ---
520# A saved credential for unattended runs, otherwise a prompt that keeps the
521# password as a SecureString.
522try {
523     if ($NXCredentialPath) {
524         $ndCredential = Import-Clixml -Path $NXCredentialPath -ErrorAction Stop
525     }
526     else {
527         $ndCredential = Get-Credential -Message "Nexus Dashboard API credentials"
528     }
529     if (-not $ndCredential) {
530         throw "No credential was given."
531     }
532
533     $ndSession = Connect-NexusDashboard -Server $NXServer `
534         -Username $ndCredential.UserName `
535         -Password $ndCredential.Password `
536         -Domain $NXDomain `
537         -TokenRefreshMinutes $TokenRefreshMinutes `
538         -TrustAllCertificates
539     Write-Log "Connected to Nexus Dashboard $NXServer as $($ndSession.Username) (domain $NXDomain)."
540} catch {
541     Write-Log "Failed to authenticate with Nexus Dashboard: $($_.Exception.Message)" -Level ERROR
542     Stop-Sessions
543     exit 1
544}
545
546# Call function "Get-NexusDashboardSwitches"
547try {
548     $switches = @(Get-NexusDashboardSwitches -Session $ndSession)
549} catch {
550     Write-Log "Failed to retrieve switches from Nexus Dashboard: $($_.Exception.Message)" -Level ERROR
551     Stop-Sessions
552     exit 1
553}
554
555# The switch list is all that is needed from Nexus Dashboard, so log out now
556# instead of keeping the token alive while the nodes are added.
557Stop-Sessions
558
559$withIp = @($switches | Where-Object { $_.IPAddress }).Count
560Write-Log "Nexus Dashboard returned $($switches.Count) switch[es], $withIp with a management IP address."
561foreach ($device in $switches) {
562     Write-Log "    switch           | name=$($device.Name) | IP=$($device.IPAddress) | model=$($device.Model) |
    serial=$($device.Serial) | fabric=$($device.Fabric) | status=$($device.Status)" -NoConsole
563}
564
565if ($switches.Count -eq 0) {
566     Write-Log "Nexus Dashboard returned no switches - nothing to process." -Level WARN

```

```

567     exit
568 }
569
570 # --- Connect to SWIS ---
571 try {
572     $swisCredential = Import-Clixml -Path $SwisCredentialPath -ErrorAction Stop
573     $swis = Connect-Swis -Hostname $SwisHost -Credential $swisCredential
574     Write-Log "Connected to SolarWinds Information Service (SWIS) on $SwisHost."
575 } catch {
576     Write-Log "Failed to connect to SWIS: $($_.Exception.Message)" -Level ERROR
577     Stop-Sessions
578     exit 1
579 }
580
581 # --- Build the list of components to process ---
582 # Build $components from the switches returned by Nexus Dashboard: the
583 # management IP address is what the node is added by, and the switch name is
584 # kept as the node caption. A switch without a valid IP address is skipped
585 # with a warning.
586 $components = foreach ($device in $switches) {
587     $address = $null
588     $ip = "$($device.IPAddress)".Trim()
589
590     if (-not $ip -or -not [System.Net.IPAddress]::TryParse($ip, [ref]$address)) {
591         Write-Log "Skipping '$($device.Name)' - no usable management IP address ('$($device.IPAddress)')." -
Level WARN
592         Write-Log "    component          | name=$($device.Name) | IP=$($device.IPAddress) | status=skipped" -
NoConsole
593         $script:stats.ComponentsSkipped++
594         continue
595     }
596
597     @{
598         IPAddress    = $address.ToString()
599         Caption      = $device.Name
600         EngineID     = $EngineID
601         SNMPVersion  = $SNMPVersion
602         DNS          = ""
603         SysName      = ""
604         Community    = $SNMPCommunity
605     }
606 }
607
608 # --- Function: Add a Node [Component] ---
609 function Add-Component {
610     param($component)
611
612     $newNodeProps = @{
613         IPAddress    = $component.IPAddress
614         EngineID     = $component.EngineID
615         Caption      = $component.Caption
616         # SNMP v2 specific
617         ObjectSubType = "SNMP"
618         SNMPVersion  = $component.SNMPVersion
619         DNS          = $component.DNS
620         SysName      = $component.SysName
621         Community    = $component.Community
622         # === default values ===
623         # EntityType   = 'Orion.Nodes'
624         # DynamicIP    = $false
625         # PollInterval = 120
626         # RediscoveryInterval = 30
627         # StatCollection = 10
628     }
629 }

```

```

630 $newNodeUri = New-SwisObject -SwisConnection $swis -EntityType "Orion.Nodes" -Properties $newNodeProps
631 $nodeProps = Get-SwisObject -SwisConnection $swis -Uri $newNodeUri
632
633 # Register the standard set of pollers for the node
634 $poller = @{
635     NetObject      = "N:" + $nodeProps["NodeID"]
636     NetObjectType   = "N"
637     NetObjectID     = $nodeProps["NodeID"]
638 }
639
640 foreach ($pollerType in @(
641     "N.Status.ICMP.Native",
642     "N.ResponseTime.ICMP.Native",
643     "N.Details.SNMP.Generic",
644     "N.Uptime.SNMP.Generic"
645 )) {
646     $poller["PollerType"] = $pollerType
647     New-SwisObject -SwisConnection $swis -EntityType "Orion.Pollers" -Properties $poller | Out-Null
648     Write-Log " poller added      | NodeID=$( $nodeProps['NodeID'] ) | pollerType=$pollerType" -NoConsole
649     $script:stats.PollersAdded++
650 }
651
652 return $nodeProps["NodeID"]
653 }
654
655 # --- Function: Discover and Add Interfaces on a Node ---
656 function Add-DiscoveredInterfaces {
657     param($nodeId)
658
659     # Discover interfaces on the node
660     $discovered = Invoke-SwisVerb $swis Orion.NPM.Interfaces DiscoverInterfacesOnNode $nodeId
661
662     if ($discovered.Result -ne "Succeed") {
663         Write-Log " Interface discovery failed for node $nodeId." -Level ERROR
664         return
665     }
666
667     # Keep only the interfaces that match -InterfaceFilter and are
668     # operationally up (ifOperStatus 1); everything else is removed before
669     # the add.
670     #
671     # The node list is materialised with @() first: RemoveChild shrinks the
672     # live XmlNodeList, and removing from it while the pipeline is still
673     # enumerating it skips nodes.
674     @($discovered.DiscoveredInterfaces.DiscoveredLiteInterface) | Where-Object {
675         $_.Caption.InnerText -notmatch $InterfaceFilter -or
676         $_.ifOperStatus -ne '1'
677     } | ForEach-Object { $discovered.DiscoveredInterfaces.RemoveChild($_) | Out-Null }
678
679     # Where-Object drops the null that an emptied node list yields: @($null)
680     # has a Count of 1, so without it a run that filtered every interface away
681     # would report one interface and still call the add verb with nothing.
682     $interfaceCount = @($discovered.DiscoveredInterfaces.DiscoveredLiteInterface | Where-Object { $_ }).Count
683
684     if ($interfaceCount -eq 0) {
685         Write-Log " No interfaces left to add for node $($nodeId) after filtering."
686         return
687     }
688
689     # Add the remaining interfaces. The node's interfaces are read before and
690     # after the add, so the log names exactly the interfaces that were added
691     # and not the ones that were already monitored.
692     $interfaceQuery = "SELECT InterfaceID, Caption FROM Orion.NPM.Interfaces WHERE NodeID = @nodeId"
693     try {
694

```

```

    $before = @(Get-SwisData -SwisConnection $swis -Query $interfaceQuery -Parameters @{ nodeId =
695 $nodeId } | ForEach-Object { $_.InterfaceID })
696
    Invoke-SwisVerb $swis Orion.NPM.Interfaces AddInterfacesOnNode @($nodeId,
    $discovered.DiscoveredInterfaces, "AddDefaultPollers") | Out-Null
697
698     $added = @(Get-SwisData -SwisConnection $swis -Query $interfaceQuery -Parameters @{ nodeId =
    $nodeId } |
699         Where-Object { $before -notcontains $_.InterfaceID })
700
701     if ($added.Count -eq 0) {
702         Write-Log " No new interfaces for node $($nodeId) - the $interfaceCount matching interface[s]
    were already monitored."
703         return
704     }
705
706     Write-Log " Added $($added.Count) interface[s] for node $($nodeId)."
707     foreach ($interface in $added) {
708         Write-Log " interface added | NodeID=$nodeId | InterfaceID=$(($interface.InterfaceID) | caption=$(
    $interface.Caption)" -NoConsole
709     }
710     $script:stats.InterfacesAdded += $added.Count
711 } catch {
712     Write-Log " Failed to add interfaces for node $($nodeId): $($_.Exception.Message)" -Level ERROR
713 }
714}
715
716# --- Main Loop: Process each component ---
717foreach ($component in $components) {
718    Write-Log "Processing component $($component.Caption) ($($component.IPAddress))..."
719
720    try {
721        # Check if node already exists
722        $existing = Get-SwisData -SwisConnection $swis -Query "SELECT NodeID FROM Orion.Nodes WHERE IPAddress
    = @ip" -Parameters @{ ip = $component.IPAddress }
723
724        if ($existing) {
725            $nodeId = @($existing)[0]
726            Write-Log " Node already exists [NodeID $nodeId], skipping add."
727            Write-Log " component | name=$(($component.Caption) | IP=$(($component.IPAddress) |
    status=exists | NodeID=$nodeId" -NoConsole
728            $script:stats.ComponentsExisting++
729        }
730        else {
731            $nodeId = Add-Component $component
732            Write-Log " Added node [NodeID $nodeId]."
733            Write-Log " component | name=$(($component.Caption) | IP=$(($component.IPAddress) |
    status=added | NodeID=$nodeId" -NoConsole
734            $script:stats.ComponentsAdded++
735        }
736
737        # Discover and add interfaces for the node (whether new or existing)
738        Add-DiscoveredInterfaces $nodeId
739
740    } catch {
741        Write-Log " Failed to process $($component.Caption) ($($component.IPAddress)): $
    ($_.Exception.Message)" -Level ERROR
742        Write-Log " component | name=$(($component.Caption) | IP=$(($component.IPAddress) |
    status=failed" -NoConsole
743        $script:stats.ComponentsFailed++
744    }
745}
746
747# --- End the SWIS session ---
748Stop-Sessions

```

```
749
750 $summary = "Summary: $($script:stats.ComponentsAdded) component[s] added, " +
751             "$($script:stats.ComponentsExisting) already existed, " +
752             "$($script:stats.ComponentsSkipped) skipped, " +
753             "$($script:stats.ComponentsFailed) failed; " +
754             "$($script:stats.InterfacesAdded) interface[s] and " +
755             "$($script:stats.PollersAdded) poller[s] added."
756 Write-Log $summary
757
758 Write-Log "Script completed."
759 Write-Log "=== Run finished ===" -NoConsole
760
761 # A scheduled task sees the run as failed when any component failed.
762 if ($script:stats.ComponentsFailed -gt 0) { exit 1 }
```